Reinforcement Learning for Blackjack

Algorithms, State Information and Finite-Deck Dynamics

Domenico Lenzerini Carlo Vincenzo Stanzione

University of Pisa

2026

Motivation and Research Questions

- How do first-visit Monte Carlo, SARSA and Q-learning compare in Blackjack?
- What changes when independent card draws become a persistent finite shoe?
- Does exposing a Hi-Lo count or the full unseen-card composition help?
- Can function approximation overcome the resulting state-space explosion?

Core idea: hold the rules fixed and vary the learner's information.

Blackjack as a Small Episodic Markov Decision Process

Rules in this repository

- Actions: *Stick* (0) or *Hit* (1).
- Dealer draws to at least 17.
- Terminal reward: $R \in \{-1, 0, +1\}$.
- Natural blackjack follows Sutton–Barto rules.
- No betting, double, split, surrender or insurance.

Standard observation

$$s = (p, d, a)$$

- p: player's current total;
- d: dealer's visible card;
- a: usable-ace flag.

A soft 18 can turn its ace from 11 to 1 after a hit; a hard 18 cannot. The flag therefore prevents two strategically different hands from being merged.

Objective and Markov property

$$\max_{\pi} \mathbb{E}_{\pi}[G_t],$$
$$G_t = \sum_{k} \gamma^k R_{t+k+1}.$$

Here $\gamma = 1$ and intermediate rewards are zero, so G_t is the final hand outcome.

With infinite sampling with replacement, (p, d, a) determines the distribution of the next observation and reward:

$$P(s', r \mid h_t, s, a) = P(s', r \mid s, a).$$

Implementation: `src/environments/blackjack.py`; rules mirror Gymnasium Blackjack-v1 with `sab=true`.

Algorithms Under Comparison: What Is Actually Updated?

First-visit Monte Carlo

Wait for the hand to finish, then use the observed return:

$$Q(s,a) \leftarrow Q(s,a) + \frac{G - Q(s,a)}{N(s,a)}.$$

No bootstrapping and no fixed α in this implementation.

SARSA: on-policy TD

Update every action using the action actually selected next:

$$y = r + \gamma Q(s', a'),$$
$$Q \leftarrow Q + \alpha(y - Q).$$

It learns the value of its exploratory policy.

Q-learning: off-policy TD

Update every action toward the greedy next action:

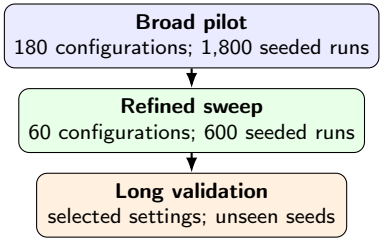
$$y = r + \gamma \max_{a'} Q(s', a'),$$
$$Q \leftarrow Q + \alpha(y - Q).$$

Behaviour may explore while the target is greedy.

All three use an ϵ-greedy behaviour policy during training and deterministic greedy actions during evaluation. Linear schedules interpolate from ϵ_{start} to ϵ_{end} over the configured fraction of training.

Experimental Pipeline: Broad Search to Fresh-Seed Validation

Stage	Protocol
Pilot sweep	$20k, 50k, 100k, 200k$ episodes; 10 seeds/configuration
Refined sweep	$100k, 200k, 500k$ episodes; 10 fresh seeds/configuration
Long validation	selected settings, larger budgets and another fresh seed set
Evaluation	frozen greedy policy; 100,000 hands per run
Ranking	mean evaluation reward; approximate 95% confidence interval across seeds



Training versus evaluation

Training explores and updates values; evaluation freezes the agent and acts greedily. Every budget starts from scratch.

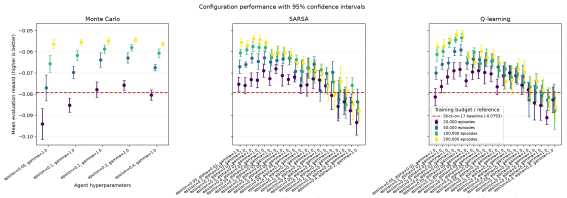
Why repeated seeds? A stochastic learner can look strong by chance; replication estimates expected performance and uncertainty.

Why fresh seeds? Selecting the maximum of many noisy scores creates optimism. New seeds test whether the winner survives that selection.

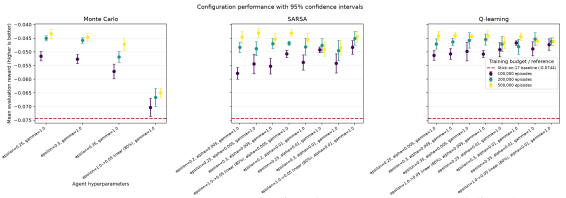
JSON → seeded runs → models/summaries → analysis. Double-DQN uses five seeds/configuration, so its conclusions are more tentative.

Standard Blackjack: Pilot Narrows, Refinement Converges

Broad pilot



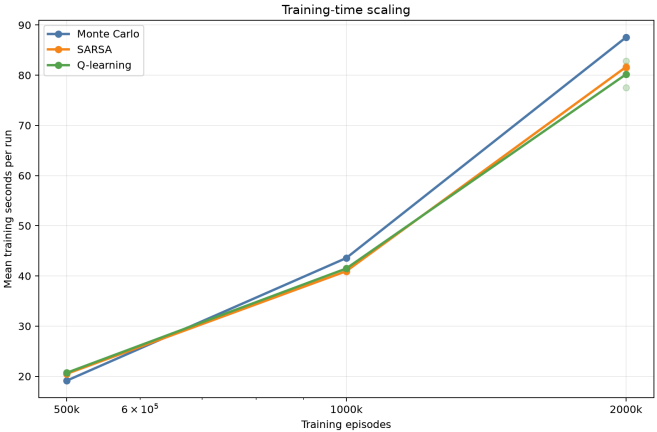
Refined sweep



Selection phase / algorithm	Episodes	Hyperparameters	Mean reward [95% CI]
Pilot: Q-learning	$200k$	$\epsilon = .30, \alpha = .01$	$-.05190$ $[-.05437, -.04944]$
Monte Carlo	$500k$	$\epsilon = .25, \gamma = 1$	$-.04342$ $[-.04522, -.04161]$
SARSA	$500k$	$\epsilon : 1 \to .05$ (80%), $\alpha = .005, \gamma = 1$	$-.04309$ $[-.04442, -.04177]$
Q-learning	$500k$	$\epsilon = .30, \alpha = .005, \gamma = 1$	$-.04403$ $[-.04529, -.04278]$

The refined means are tightly grouped; their paired differences are inconclusive. The apparent winner still requires fresh-seed validation.

Standard Blackjack: Fresh-Seed Long-Budget Validation



Measured result

	Budget	Reward
MC	$2M$	$-.04790$
SARSA	$1M$	$-.04646$
Q-learn.	$500k$	$-.04664$

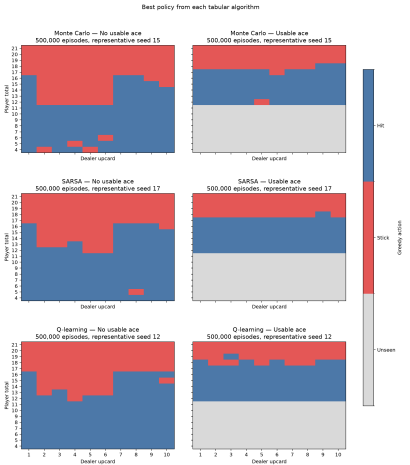
Winner SARSA: $\epsilon : 1 \to .05$ over 80%, $\alpha = .005$; 95% CI $[-.04738, -.04553]$.

The dashed benchmark used throughout is stick-on-17, not random play.

Standard Policies: Behaviour by Visible Game State

How to read it

- Rows: player total.
- Columns: dealer upcard.
- Red: Stick; blue: Hit.
- Left/right panels separate hard hands and hands with a usable ace.
- Each row of panels uses a representative seed from that algorithm's best refined configuration.



Finite-Shoe Dynamics: What Changes?

Infinite deck

$$P(C_{t+1} = c) \text{ is constant.}$$

Cards are sampled with replacement, so the three-value observation is Markov.

Finite environment in this code

- six decks (312 cards);
- 75% penetration (cut after about 234 dealt cards);
- the persistent shoe is reshuffled only before a new hand;
- dealer hole card stays hidden until reveal.

Implementation: `src/environments/finite_blackjack.py`.

Hidden observation

$$s = (p, d, a)$$

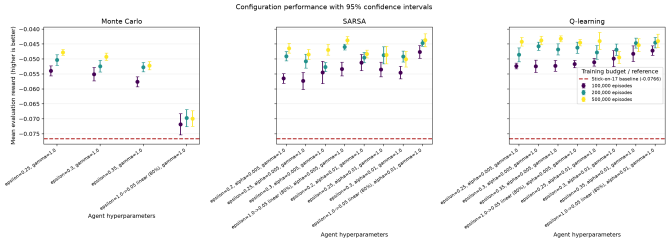
The same visible state can now imply different next-card distributions because the unobserved shoe composition changes across hands.

Theoretical consequence

For the agent, finite-hidden Blackjack is a partially observable decision process: the visible tuple aliases multiple underlying shoe states.

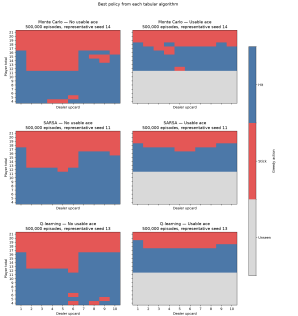
Finite Hidden: Standard-Level Reward and a Stable Visible Policy

Refined-sweep performance



Best	Configuration	Reward [95% CI]
MC	$500k, \epsilon = .25$	$-.04780$ [$-.04878, -.04681$]
SARSA	$500k, \epsilon : 1 \to .05, \alpha = .01$	$-.04368$ [$-.04584, -.04152$]
Q-learn.	$500k, \epsilon = .35, \alpha = .005$	$-.04320$ [$-.04418, -.04222$]

Best learned visible policy



The familiar Hit/Stick boundary remains. Under fixed stakes, six decks and 75% penetration, hidden shoe composition changes decisions less than hand total and dealer card do.

Interpretation Q-learning has the largest mean (-0.04320), but its difference from SARSA is inconclusive; standard-level reward does not make the observation Markov.

Adding Information: Hi-Lo Count State

Observed state

$$s = (p, d, a, \widehat{TC})$$

$$2\text{–}6 : +1, \qquad 7\text{–}9 : 0, \qquad 10, A : -1.$$

The running count is updated only for publicly observed cards; the dealer hole card enters when revealed.

Repository-specific true-count bucket

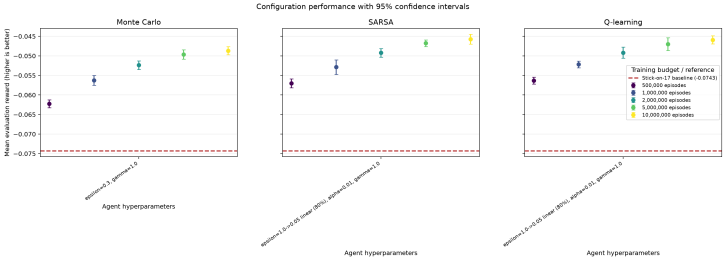
$$\widehat{TC} = \mathrm{clip}_{[-20,20]}\left(\mathrm{int}\,\frac{RC}{\max(D_{remain}, 0.25)}\right).$$

This compact statistic distinguishes some shoes that looked identical in the hidden model, but it also multiplies the tabular state space by up to 41 buckets.

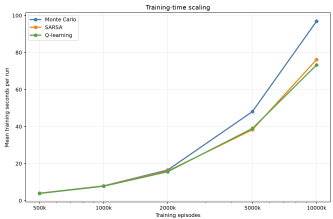
Because stakes are fixed and only Hit/Stick is available, this experiment tests whether count information improves playing decisions–not the betting advantage normally associated with card counting.

Hi-Lo: More Samples, Little Reward Gain

Mean evaluation reward



Training time



At $10M$ episodes

MC	96.85 s
SARSA	76.15 s
Q-learning	73.26 s

Best result SARSA at $10M$, $\epsilon : 1 \to .05$ over 80%, $\alpha = .01$: -0.04571 $[-0.04696, -0.04446]$. Q-learning: -0.04589; MC: -0.04867.

More data helps all three, but does not establish a Hi-Lo advantage over simpler observations. Runtime is approximately linear and hardware-specific.

Exact Unseen-Card Composition and State Explosion

Composition observation

$$s = (p, d, a, n_A, n_2, \ldots, n_9, n_{10}).$$

The ten counts describe cards not yet publicly observed. They include the hidden dealer card until it is revealed; conditional on these counts, its identity remains uncertain.

The representation is highly informative and 13-dimensional, but exact count vectors almost never repeat across independently shuffled shoes.

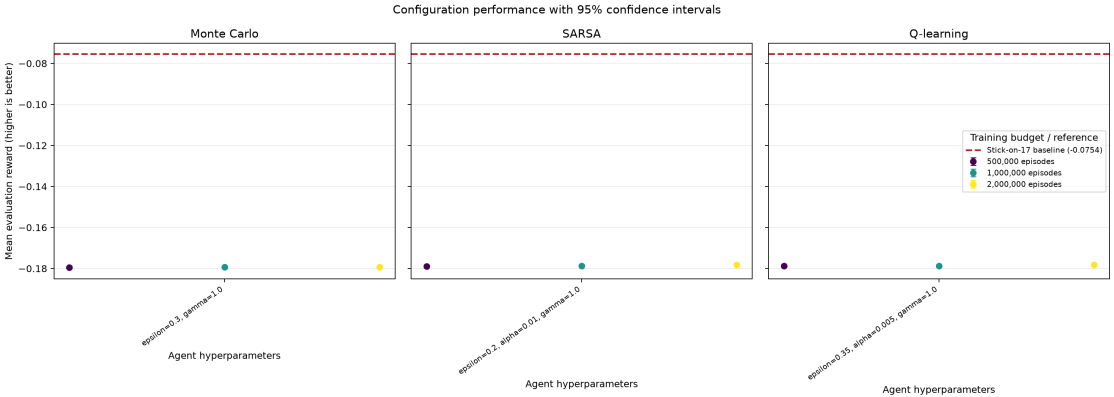
Tabular consequence

The final agents store about $2.69M$ exact states, yet approximately 96.5% of evaluation decisions still encounter a state absent from that run's Q-table.

Why reward collapses

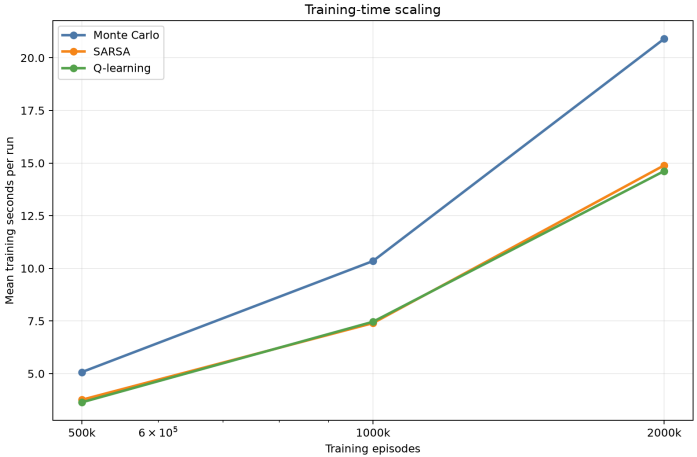
Unseen action values are both zero and deterministic evaluation breaks ties toward action 0 (Stick). Thus sparse coverage becomes a behavioural bias, not merely a memory problem.

Composition-Aware Tabular Agents: Performance Collapse



Best measured result SARSA at $2M$ episodes, $\epsilon = .20, \alpha = .01$: -0.17819, 95% CI $[-0.17866, -0.17773]$. Stick-on-17 comparator: -0.07544.

Composition-Aware Tabular Agents: Training Time



At $2M$ episodes

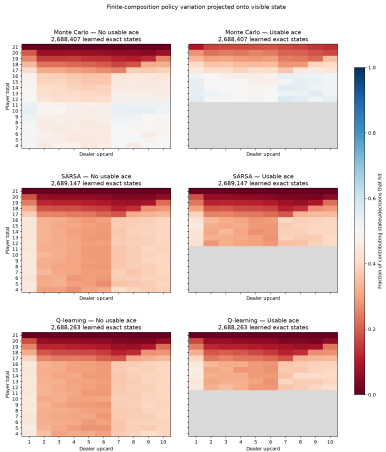
MC	20.90 s
SARSA	14.89 s
Q-learning	14.62 s

Key distinction Runtime scales acceptably; generalisation does not.

Composition Tabular Policy: Projection over Hidden Dimensions

Meaning of a cell
For one visible tuple (p, d, a), many exact count vectors exist. The colour is the fraction of those learned exact states whose greedy action is Hit.

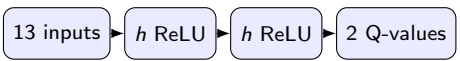
- (0): always Stick among contributors;
- (1): always Hit;
- near (0.5): composition-dependent disagreement;
- gray: no contributing state.



Double DQN: Function Approximation for Composition States

Normalized 13-feature input

$$\left[\frac{p}{21}, \frac{d}{10}, a, \frac{n_A}{24}, \ldots, \frac{n_9}{24}, \frac{n_{10}}{96}\right]$$



Implemented stabilisers

- replay buffer 100,000, batch 64 or 128;
- Adam optimiser and Huber loss;
- gradient norm clipping at 10;
- one update every 4 actions after 1,000 actions;
- target network copied every 1,000 actions.

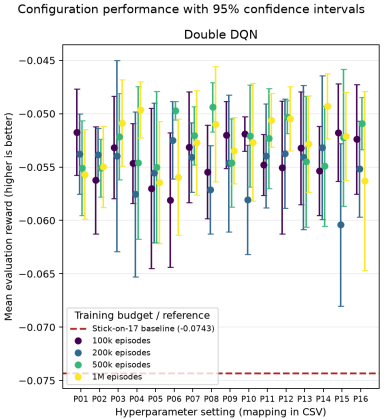
Double target

$$a^* = \arg\max_a Q_{\text{online}}(s', a),$$

$$y = r + \gamma Q_{\text{target}}(s', a^*).$$

This separates action selection from evaluation, following van Hasselt, Guez and Silver (AAAI 2016).

Double DQN: Refined-Sweep Performance



Best configuration $1M$ episodes; $\epsilon : 1 \rightarrow .01$ over 100%; $\eta = .001$; batch 64; hidden width 128.

Mean reward	-0.04929
95% CI	$[-0.05234, -0.04624]$
Win rate	42.83%
Mean training time	546.00 s
Network parameters	18,562

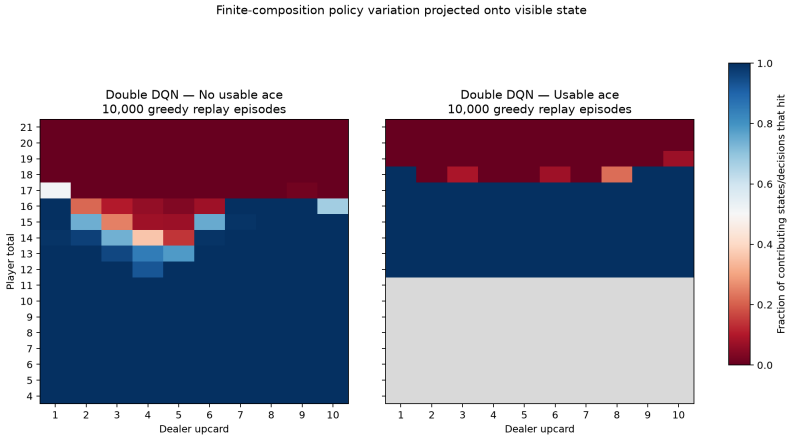
Only five seeds and selection on the displayed sweep: promising evidence, not a final unbiased estimate.

Double-DQN Policy: Visible-State Projection

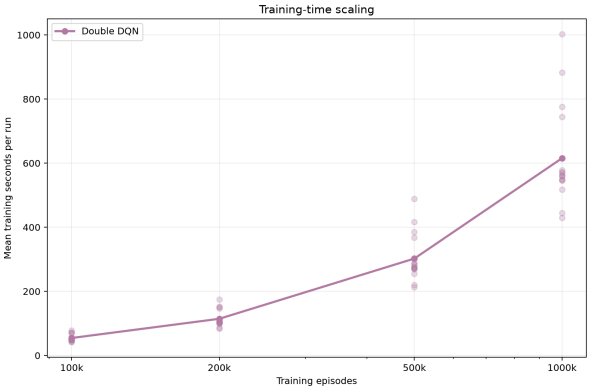
The policy is deterministic for the full 13-feature input, but not necessarily for only (p, d, a).

The plot replays (10,000) greedy episodes, groups decisions by visible cards and reports the fraction that choose Hit.

Mixed colours therefore indicate composition-dependent action changes, not random evaluation actions.



Double DQN: Computational Cost



Selected (1M) run

Mean time (546.00) seconds per training seed.

Each transition can trigger neural inference; every fourth action can add a replay-buffer gradient update.

The trade-off is clear: much stronger composition-state generalisation, substantially higher CPU cost.

Conclusions and Limits

What the experiments support

- Standard tabular methods learn similar strong Hit/Stick policies; fresh validation favours SARSA only slightly.
- Hidden finite-shoe dynamics do not materially damage aggregate reward under the tested rules.
- Hi-Lo information increases sample demand without a clear reward advantage here.
- Raw exact composition breaks tabular coverage; Double DQN recovers performance through generalisation.

What remains to establish

- repeat the selected DQN with 10–20 fresh seeds;
- run paired, pre-specified cross-environment comparisons;
- measure memory and learning curves, not only final reward and time;
- test compact composition features or recurrent/belief-state agents;
- expand the environment before drawing conclusions about Double, Split, Insurance or betting.

A negative mean reward is expected in fixed-stake casino Blackjack. This project compares policies under a deliberately restricted action set; it is not a full casino-strategy simulator.

References and Reproducibility

Farama Foundation.

Gymnasium Blackjack-v1 documentation.

https://gymnasium.farama.org/environments/toy_text/blackjack/

R. S. Sutton and A. G. Barto.

Reinforcement Learning: An Introduction, 2nd ed., Example 5.1.

http://incompleteideas.net/book/RLbook2020.pdf

H. van Hasselt, A. Guez and D. Silver.

"Deep Reinforcement Learning with Double Q-Learning," AAAI, 2016.

https://doi.org/10.1609/aaai.v30i1.10295

All displayed numbers come from versioned JSON summaries in `results/`; all figures are generated by `src/analyze.py`.